PROGRAMMABLE DATA PLANE AND P4

Rafael Olim 93326

Diogo Ribeiro 102484

1. INTRODUCTION

This project explores the concept of programmable data planes and their practical implementation using the P4 programming language, along with supporting tools such

as Mininet, BMv2, and P4Runtime. Unlike traditional fixed function forwarding hardware, programmable data planes enable network operators to define how packets are parsed, processed, and forwarded, providing flexibility for custom protocols, traffic filtering, and advanced features without requiring new hardware.

The lab is organized into two main tasks. The first introduces P4 through the implementation of basic IPv4 forwarding, while the second extends this foundation with a packet filtering and marking mechanism to detect and flag suspicious traffic. Together, these exercises demonstrate how P4 empowers flexible, customizable, and innovation-driven packet processing, showcasing the practical benefits of programmable data planes in modern networking.

2. OBJECTIVES

The main objective of this lab is to develop an understanding of programmable data planes and their role in modern networking. Specifically, the lab aims to provide familiarity with the Mininet environment, BMv2 software switches, and the P4Runtime interface, while introducing the structure and syntax of the P4 programming language. Through the implementation of a basic IPv4 forwarding mechanism, students will learn how to parse packet headers, update fields, and forward traffic within a pod-topology. Building on this foundation, the lab also seeks to extend packet processing with a filtering and marking mechanism, enabling the identification and handling of suspicious traffic based on source IP and port. By the end of the lab, students are expected to have gained practical skills in programming, testing, and validating programmable data plane applications using P4 and related tools.

3. EXPERIMENTAL METHODOLOGIES

The experimental environment for all tasks is based on **Mininet**, which emulates the network topology within a virtualized platform. The switches in the topology are instantiated as **BMv2** software switches, capable of executing **P4** programs, while **P4Runtime** provides runtime control to install forwarding and filtering rules. All experiments are performed inside a preconfigured **Ubuntu VM** environment provided for the lab.

3.1. TASK 1 – BASIC FORWARDING

In this task, a pod-topology is deployed in **Mininet**, consisting of 4 hosts and 4 switches connected as a single pod of a fat-tree topology. Each switch runs the *basic.p4* program on **BMv2**, and the control plane populates the forwarding tables with static rules using **P4Runtime**.

As in the first lab, we began by analysing the provided code to understand its functionality and identify what was missing to achieve proper packet forwarding.

Since the headers were already defined in the P4 program and the tables populated by the controller, we implemented the missing forwarding logic. We first ensured proper parsing of the IPv4 header, then configured the forwarding table to apply only when the header was correctly parsed, allowing the longest prefix match (LPM) on the destination address to work correctly. The missing ipv4_forward action was added to behave like a normal layer 3 switch (setting the correct egress port, updating MAC addresses, and decrementing the TTL). Finally, the deparser was updated to emit packets with the modified headers in the correct order.

3.2. TASK 2 – MARK AND FILTER

For this task, the same pod-topology is reused, with additional logic to filter and mark suspicious traffic. Packets carry a custom filter header, which switches use to flag packets based on inspection of the source IP and (UDP) port. The controller script (*mycontroller.py*) will use **P4Runtime** to install forwarding and filtering rules on the switches. The *filter.p4* program running on each switch is intended to provide the necessary table structures and actions to mark packets appropriately and forward them through the topology.

Leveraging the knowledge acquired in the previous task, we began by defining the filter and UDP headers. The filter header contained two fields of 1 byte each: *susp* and *proto*. The UDP header was defined according to its standard structure, with four fields of 2 bytes each, as implemented in our program.

The parser state machine was constructed based on the header order and specific protocol numbers in the IPv4 and filter headers, as specified.

A filter table was created to match the exact source IP and UDP source port and, if a match was found, apply the marking action by setting the *susp* field to 1. The default action left packets unchanged. In the controller, the table was populated with entries from the *susp.txt* file, specifying which packets should be marked as suspicious. Correct operation required all relevant headers to be parsed; in practice, checking the UDP header was sufficient, since preceding headers are parsed sequentially by the state machine. Therefore, the table was applied only to those packets. The forwarding table and its rules were already defined similarly to what was done in the first task, so we did not modify this part.

Finally, in the deparser, headers were converted back to raw bytes in the correct order by explicitly emitting each header, ensuring proper packet reconstruction for forwarding.

4. Result Analysis

4.1. Task 1 – Basic Forwarding

Initially, pings between any hosts did not work because the IPv4 headers were not correctly parsed and the forwarding action had not been implemented.

After implementing the solution, we verified its correctness. The ***pingall*** command confirmed connectivity between all hosts, and packet-level inspection demonstrated correct forwarding behaviour.

For example, an ICMP packet sent from h1 (10.0.0.1) to h2 (10.0.0.2) arrived at switch s1 on interface 1 (ingress port) with MAC address 08:00:00:00:01:00 and TTL 64 (shown in **Fig1**). The switch then processed the packet according to the P4 program, forwarding it out of interface 2 (egress port), where the packet was captured (as observed in **Fig2**). The source MAC was updated to the MAC of the switch's ingress port, the destination MAC set to h2's MAC, and the TTL decremented from 64 to 63, representing a single hop.

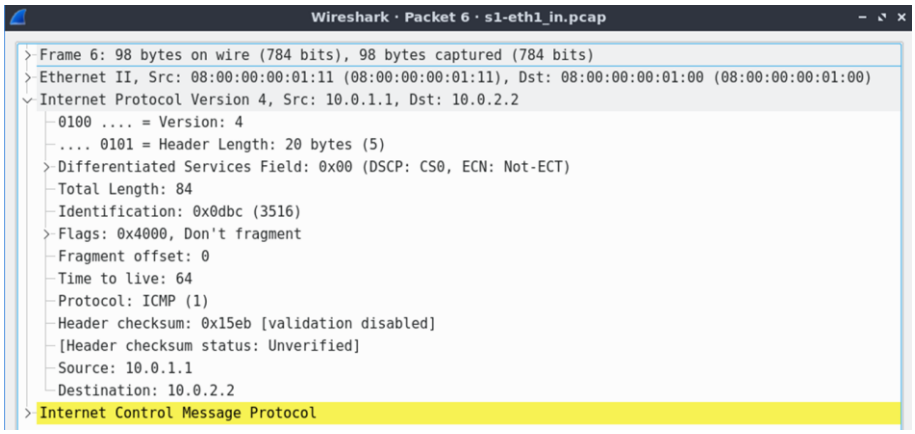


Fig1 - ICMP packet entering s1 on ingress port (eth1)

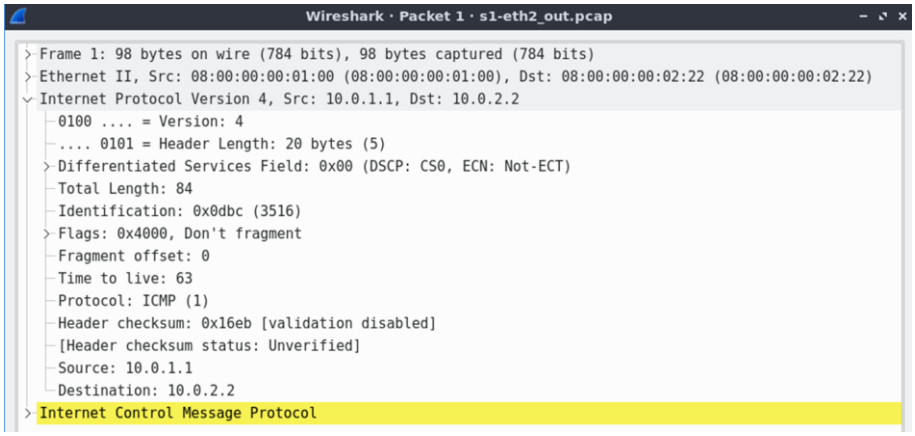


Fig2 - ICMP packet leaving s1 on egress port (eth2)

All other packets showed consistent behaviour, and the ICMP replies followed the correct paths, demonstrating that the forwarding logic was correctly implemented.

4.2. TASK 2 – MARK AND FILTER

To test the correctness of the implementation, we ran the ***make run*** command, followed by initializing the controller in a separate terminal to populate the switch tables. The accompanying test script automatically generated traffic between the hosts, allowing us to verify the results. When no mismatches were observed, we concluded that the implementation was functioning correctly.

As an example, consider two packets sent from host 1 to host 3. To find these packets we inspected the logs on h3 using ***cat h3.txt | grep -w "10.0.1.1"*** This filtered only the packets originating from h1, allowing us to identify at least one suspicious and one non-suspicious packet. Next, we analysed the traffic in **Wireshark** using the filter ***ip.proto == 146 && !(ip.proto == 1)***. This corresponds to the custom IPv4 protocol containing the filter header followed by the UDP header. Finally, we traced the packets entering s1 on interface 1 (connected to h1) and exiting s2 on interface 1 (connected to h3).

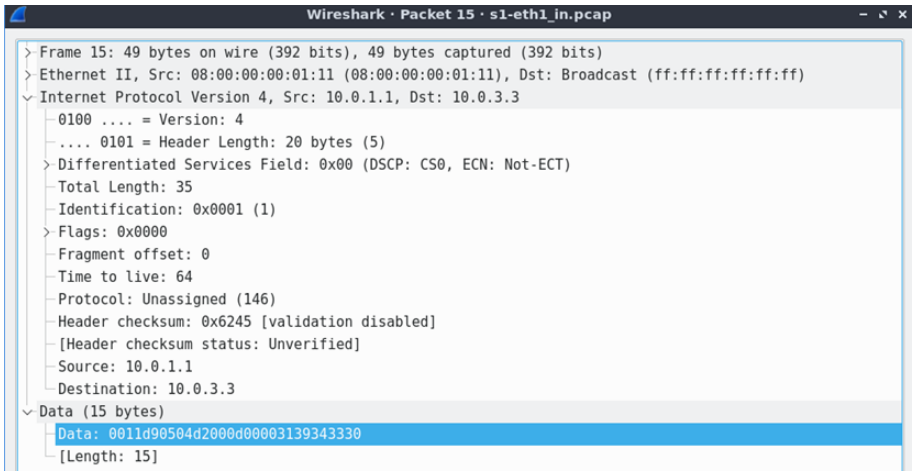


Fig3 - Non-suspicious packet entering s1 on ingress port (eth1)

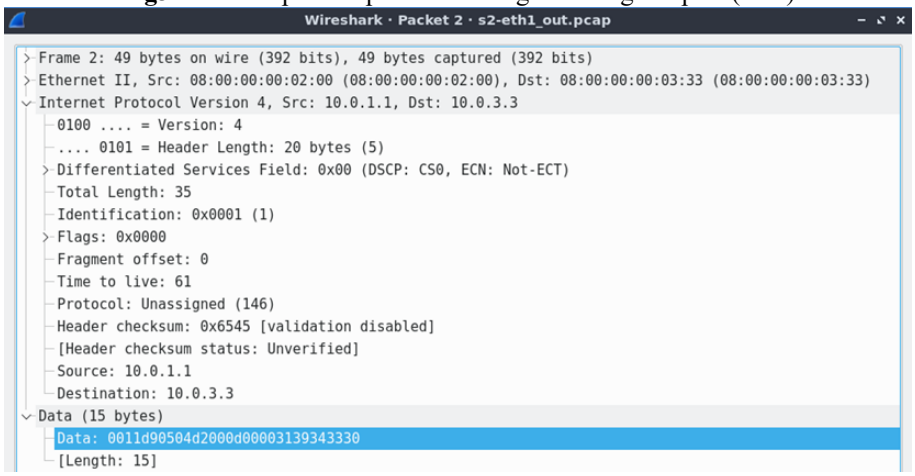


Fig4 - Non-suspicious packet leaving s2 on egress port (eth1)

The first packet arriving at s1 (**Fig3**) was not suspicious, as confirmed by inspecting the IPv4 payload. According to the specification, the first 2 bytes correspond to the filter header, the next 8 bytes to the UDP header, and the remainder to the actual payload. In the UDP header, the source port (first 2 bytes: 0xd905) converts to decimal 55557 and, since the pair (10.0.1.1, 55557) is not in the suspicious list, the packet should be processed as non-suspicious. In **Fig4**, the first byte of the filter header indicates the packet was not marked, confirming correctness. Additionally, the MAC addresses and TTL were correctly updated, reflecting 3 hops according to the topology. Furthermore, the second byte of the filter header (0x11) corresponds to protocol 17 (UDP), confirming that we were analysing the expected packet type.

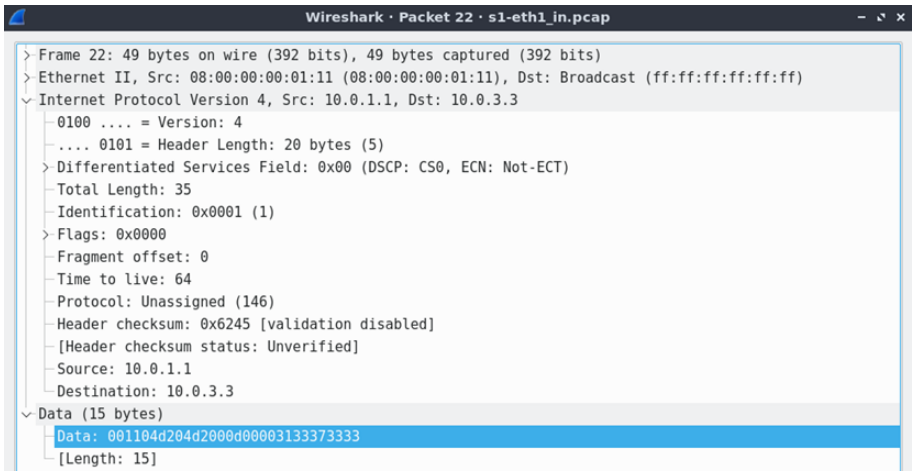


Fig5 - Suspicious packet entering s1 on ingress port (eth1)

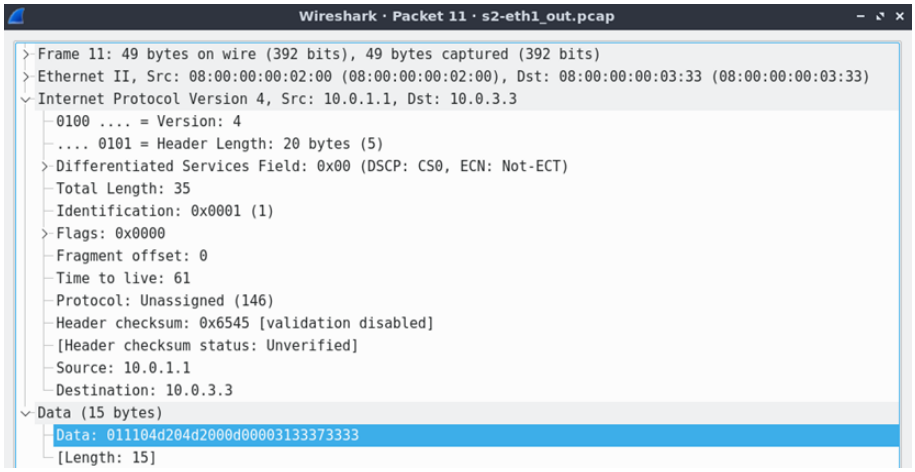


Fig6 - Suspicious packet leaving s2 on egress port (eth1)

The second packet reaching s1 (**Fig5**) was suspicious. Its source port (0x04d2 converting to 1234) combined with source IP 10.0.1.1 matched an entry in the suspicious list. This time, the packet was marked (as observed in **Fig6**) accordingly in the filter header, while MAC addresses and TTL were updated correctly, demonstrating that the filtering logic works as intended.

Through these observations, we verified that the implementation correctly distinguishes suspicious from non-suspicious packets, marking them appropriately while preserving normal forwarding behaviour. This approach highlights the potential of programmable switches for flexible traffic monitoring and network control, even in more complex topologies.

During the tests, an unexpected behaviour was observed on the first egress port (i.e. the first processing of a packet arriving from a host), because the hosts always used the

broadcast address (ff:ff:ff:ff:ff:ff) as the ethernet destination. As a result, the switches incorrectly used the broadcast address as the source MAC instead of their own interface MAC. We investigated the issue but found it difficult to resolve due to limitations of the BMv2 V1Model architecture, which does not provide automatic access to the ingress port's MAC address in the data plane. Correcting this would have required hardcoded rules or additional tables. Given that this behaviour did not affect overall functionality, since the switches primarily operate at layer 3 to forward packets to the hosts, we proceeded without fixing the issue.

5. CONCLUSIONS

5.1. TASK 1 – BASIC FORWARDING

This task demonstrated that implementing IPv4 forwarding in P4 enables a software switch to behave like a standard Layer 3 device. Unlike legacy fixed-function switches, however, the P4-based approach offers greater flexibility, since the forwarding logic is defined in software and can be extended or modified to support new protocols and behaviours beyond basic L3 forwarding.

5.2. TASK 2 – MARK AND FILTER

This task demonstrated that implementing traffic filtering and marking in P4 enables the switch to selectively identify and flag suspicious packets while still performing normal forwarding. Unlike legacy fixed-function switches, the P4-based approach provides unmatched flexibility: the filtering logic is fully programmable, allowing easy adaptation to new detection criteria, custom headers, or emerging protocols without changing the hardware. This programmability also enables real-time traffic monitoring, dynamic policy updates, and fine-grained network control, which are difficult or impossible on traditional switches.

Overall, the task highlights how P4 switches can combine standard L3 forwarding with advanced packet inspection and security features, offering a powerful platform for flexible, programmable, and future-proof network infrastructures.

6. APPENDIX

The following topics describe how the annexed files (the ones located in the same directory / zip as this pdf report) were placed and modified based on the provided base code:

- The *basic.p4* file replaces the original one in the *tutorials/exercises/basic/* directory.
- The provided *filter.zip* files were extracted into *tutorials/exercises/filter/*. The *mycontroller.py* and *filter.p4* files were then modified and replaced the originals in the same directory, while all other files remained unchanged.